

PSO-BASED RANDOM FOREST HYPERPARAMETER OPTIMIZATION

A Multi-Dataset Experimental Study Using
Iris, **Digits**, and **Heart Disease** Classification



ANUSH RAMESH



CB.EN.U4ELC23005



EEE DEPARTMENT



AMRITA VISHWA VIDYAPEETHAM, COIMBATORE



Optimization



Machine Learning



Performance



PROBLEM STATEMENT



Random Forest is a powerful ensemble machine-learning algorithm, but its **performance depends** on selecting **suitable hyperparameters**.



IMPORTANT HYPERPARAMETERS



n_estimators

Number of trees in the forest



max_depth

Maximum depth of the trees



min_samples_split

Minimum number of samples required to split a node



TRADITIONAL MANUAL TUNING



Requires trial-and-error

Many combinations must be tried manually



Is time-consuming

Needs significant time and computational cost



May not explore the search space efficiently

Risk of missing better configurations



Depends heavily on user experience

Results vary with the expertise of the user



There is a need for an **automated**, **efficient**, and **intelligent** hyperparameter optimization approach to achieve **better performance** with less human effort.

MOTIVATION





OBJECTIVES

01



Build a reproducible Random Forest baseline.

Create a strong and consistent baseline for fair comparison with optimized models.

02



Implement Particle Swarm Optimization from scratch.

Develop a PSO algorithm including initialization, fitness evaluation, velocity and position updates.

03



Use PSO to optimize Random Forest hyperparameters.

Leverage PSO to search for the best combination of Random Forest hyperparameters within the defined bounds.

04



Evaluate candidate configurations using cross-validation.

Use k-fold cross-validation to reliably estimate performance of each candidate solution.

05



Maintain strict train/validation/test separation.

Ensure no data leakage and maintain a proper separation for unbiased evaluation.

06



Compare optimized Random Forest against the baseline.

Evaluate whether PSO-optimized configurations outperform or match the baseline model.

07



Analyze PSO convergence.

Study the convergence behavior of the swarm and how fitness improves over iterations.

08



Evaluate stability across multiple random seeds.

Assess the consistency and robustness of PSO results using multiple random seeds.

09



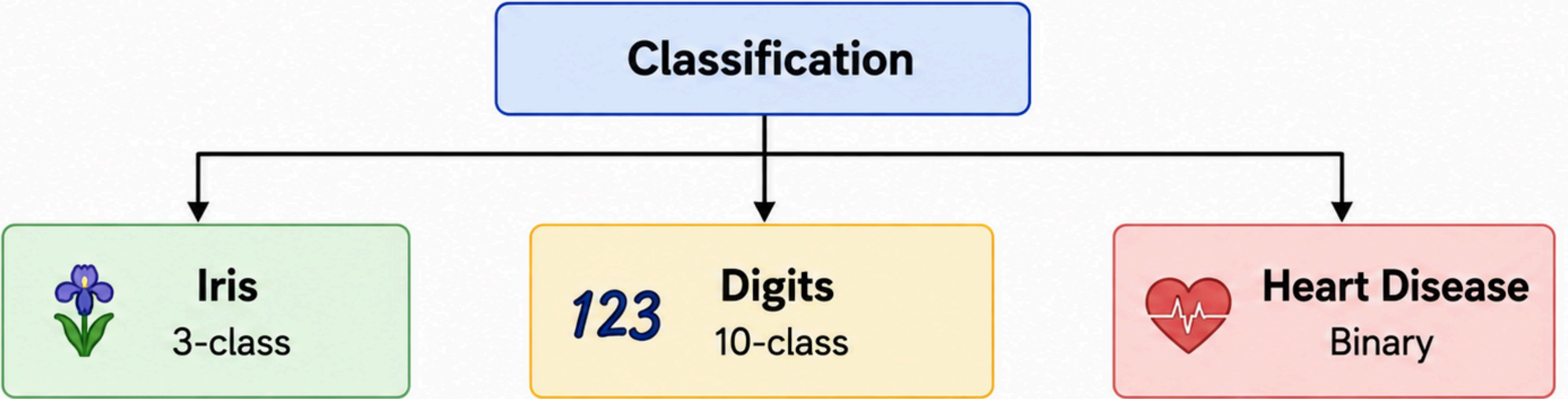
Test the approach across multiple classification datasets.

Validate the effectiveness and generalizability of the approach on multiple real-world datasets.

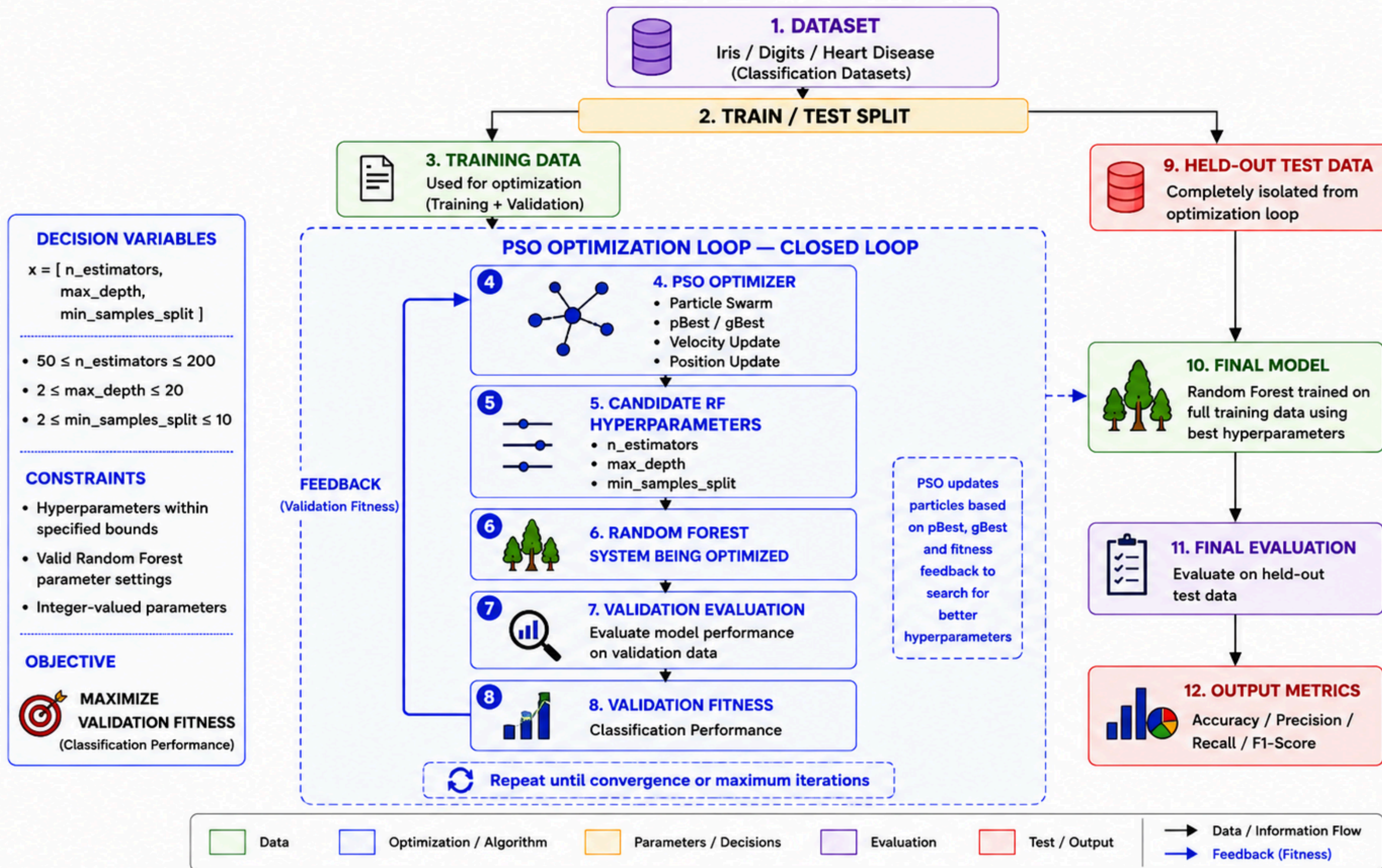
DATASET DESCRIPTION

Datasets

Dataset	Task	Characteristics
 Iris	Classification	Small, 3-class dataset
 Digits	Classification	10-class handwritten digit recognition
 Heart Disease	Classification	Binary real-world classification



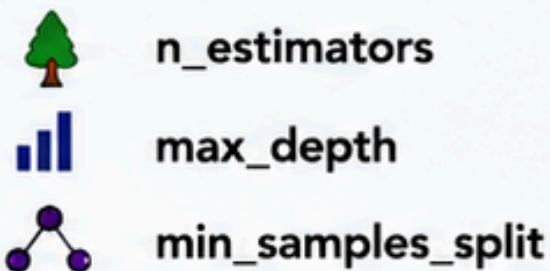
SYSTEM ARCHITECTURE



OPTIMIZATION FORMULATION

DECISION VARIABLES

PARTICLE



$x = [n_estimators, max_depth, min_samples_split]$

- n_estimators**
→ Number of trees in the Random Forest
- max_depth**
→ Maximum depth of each decision tree
- min_samples_split**
→ Minimum samples required to split an internal node



OPTIMIZATION PROBLEM

Find x^* that maximizes
VALIDATION FITNESS

$x = [n_estimators, max_depth, min_samples_split]$

maximize $F(x)$

where:

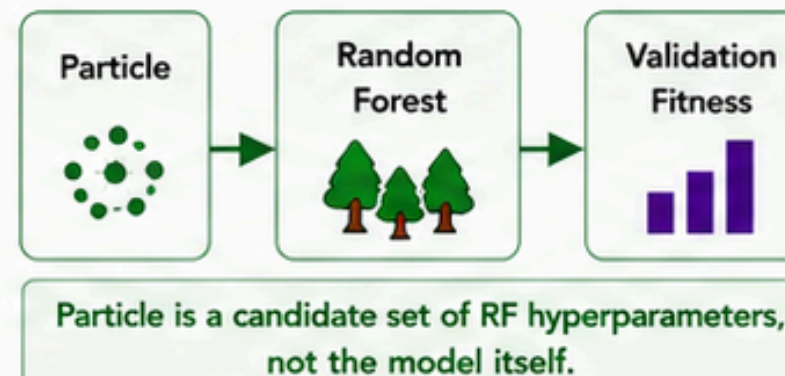
$F(x) = \text{Validation Fitness of Random Forest}(x)$



CANDIDATE PARTICLE (EXAMPLE)

$[120, 8, 4]$

120 → n_estimators
8 → max_depth
4 → min_samples_split



SEARCH SPACE / CONSTRAINTS

$50 \leq n_estimators \leq 200$

$2 \leq max_depth \leq 20$

$2 \leq min_samples_split \leq 10$

Parameter	Range	Type
n_estimators	50 – 200	Integer
max_depth	2 – 20	Integer
min_samples_split	2 – 10	Integer

FITNESS EVALUATION & FEEDBACK LOOP



TEST SET ISOLATION

TRAINING / VALIDATION



PSO FITNESS

(Used for Optimization)

HELD-OUT TEST SET



FINAL EVALUATION

(Used Once After Optimization)



Test set is reserved for final evaluation — **NOT** used as PSO fitness.

WHY PARTICLE SWARM OPTIMIZATION?



PSO searches the Random Forest hyperparameter space using a **swarm of candidate solutions** and improves them using **individual** and **global** experience.



WHY PSO?



1. Population-Based Search

→ Evaluates multiple candidate solutions.



2. Derivative-Free

→ Does not require gradients.



3. Suitable for Bounded Search

→ Works naturally with defined hyperparameter ranges.



4. Exploration + Exploitation

→ Combines individual and swarm experience to search for better configurations.



Why PSO for Random Forest?

Random Forest hyperparameters form a bounded search space. PSO can explore multiple candidate configurations and use validation fitness to guide the search.



WHAT IS A PARTICLE?

PARTICLE



n_estimators



max_depth



min_samples_split

$x = [n_estimators, max_depth, min_samples_split]$

Particle A (Example)

[120, 8, 4]

120 → n_estimators

8 → max_depth

4 → min_samples_split

Particle = Candidate Random Forest Hyperparameters

Swarm of Candidate Solutions

Particle A
[120, 8, 4]



Particle B
[180, 15, 3]



Particle C
[75, 5, 7]



HOW PSO SEARCHES FOR BETTER HYPERPARAMETERS

1. Initialize Swarm

- Random candidate hyperparameter sets within defined bounds

2. Evaluate Fitness

- Train Random Forest
- Compute validation fitness

3. Update pBest

- Best position found by each particle

4. Update gBest

- Best position found by the entire swarm

5. Velocity + Position Update

- Move particles towards pBest and gBest
- Explore new regions in the search space

Search improves with each iteration

6. Repeat

- Iterate until convergence or maximum iterations



PSO FEEDBACK LOOP

Candidate Particle
[n_estimators, max_depth, min_samples_split]

Random Forest



Validation Evaluation



(Validation Fitness)

Fitness Value



Update pBest / gBest



Next Iteration



(Better Hyperparameters)

Optimization Feedback



Search Space / Constraints

$50 \leq n_estimators \leq 200$

$2 \leq max_depth \leq 20$

$2 \leq min_samples_split \leq 10$

PARTICLE SWARM OPTIMIZATION ALGORITHM



PSO searches the solution space using **pBest** and **gBest** to guide particle movement toward the best solution.

1. PARTICLE & SWARM

PARTICLE

= One candidate solution

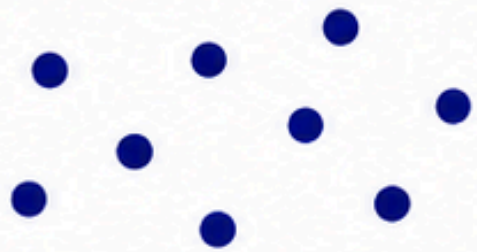
$$x_i = [x_1, x_2, x_3]$$

For this project:

$$x_i = \begin{bmatrix} \text{n_estimators,} \\ \text{max_depth,} \\ \text{min_samples_split} \end{bmatrix}$$

SWARM

= Collection of particles



Swarm = Population of Candidate Solutions

2. PSO ITERATIVE ALGORITHM

- 1 INITIALIZE SWARM**
Randomly initialize particle positions and velocities.
- 2 EVALUATE PARTICLES**
Calculate the objective/fitness value for each particle.
- 3 UPDATE pBest**
Compare current fitness with the particle's previous best. If current solution is better:
 $\text{pBest} \leftarrow \text{current position}$
- 4 UPDATE gBest**
Find the best solution among all particles.
 $\text{gBest} \leftarrow \text{best particle position}$
- 5 UPDATE VELOCITY**
Calculate the new velocity using:
– Previous velocity – **pBest** – **gBest**
- 6 UPDATE POSITION**
Move the particle:
 $x_i(t+1) = x_i(t) + v_i(t+1)$
- 7 REPEAT**
Evaluate the new positions and repeat until the stopping condition is reached.

VELOCITY UPDATE

$$v_i(t+1) = wv_i(t) + c_1r_1(\text{pBest}_i - x_i(t)) + c_2r_2(\text{gBest} - x_i(t))$$

INERTIA
 $wv_i(t)$
→ Maintains previous movement

COGNITIVE
 $c_1r_1(\text{pBest}_i - x_i)$
→ Moves toward the particle's own best

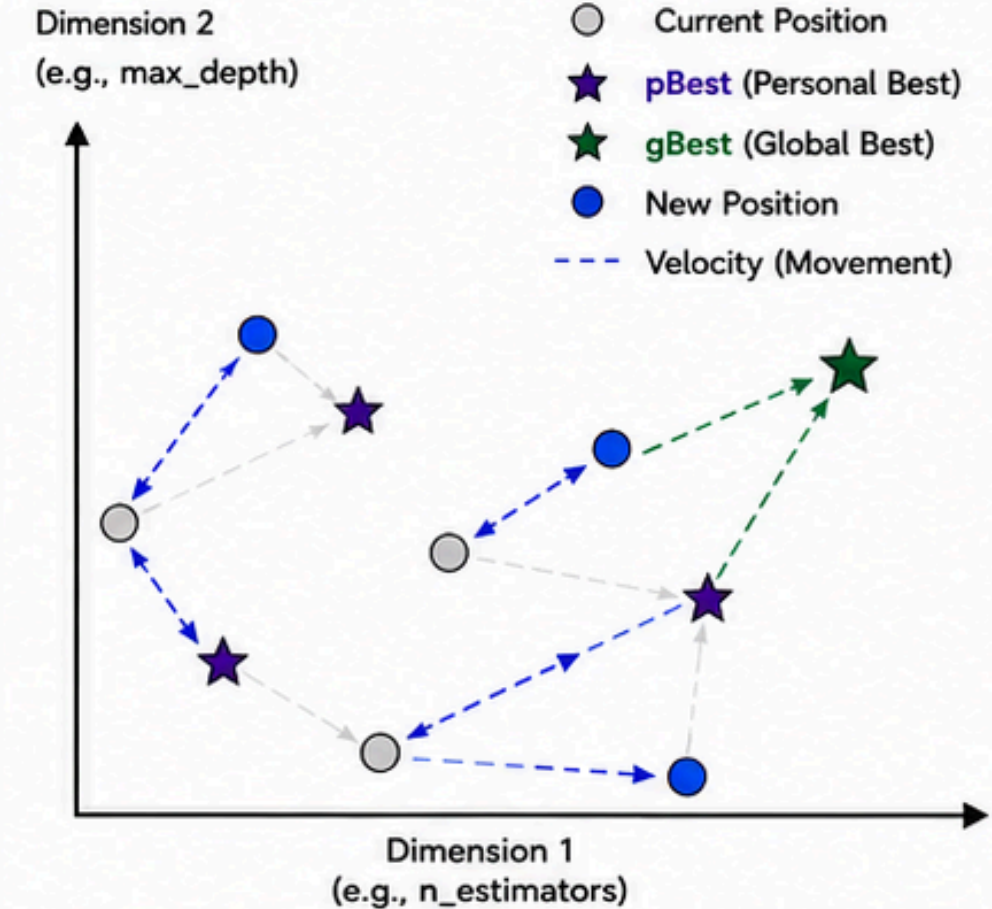
SOCIAL
 $c_2r_2(\text{gBest} - x_i)$
→ Moves toward the swarm's best

POSITION UPDATE

$$x_i(t+1) = x_i(t) + v_i(t+1)$$

New position = New candidate solution

3. PARTICLE MOVEMENT (2D SEARCH SPACE)



Particle movement is influenced by:

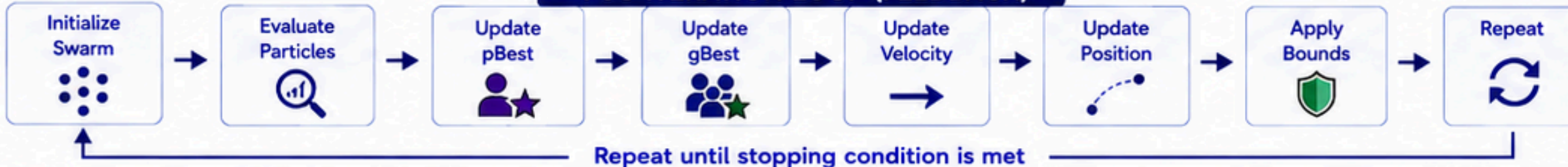
- Current velocity
- **pBest** (individual experience)
- **gBest** (swarm experience)

pBest vs gBest

pBest (Personal Best)
Best position found so far by an individual particle.

gBest (Global Best)
Best position found so far by the entire swarm.

PSO ITERATION LOOP (FEEDBACK)



Key Points:

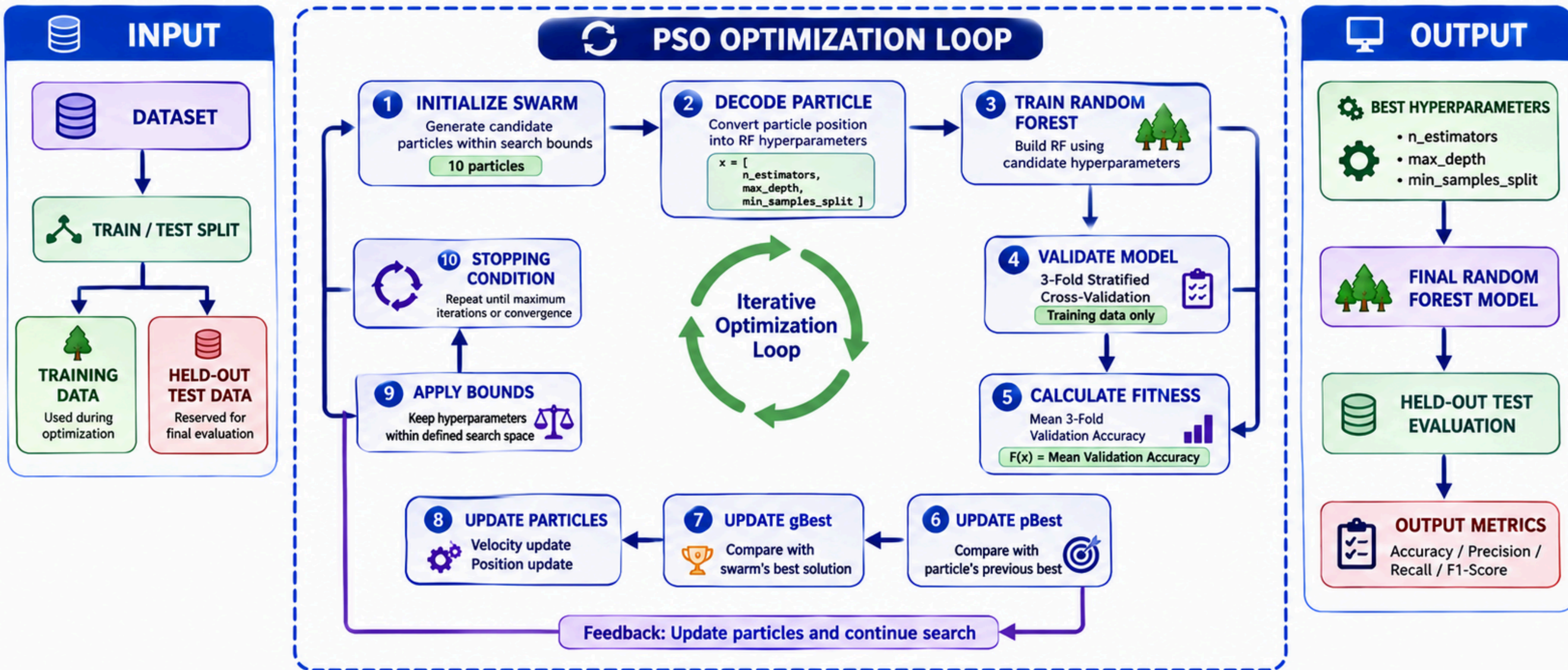
- PSO is a population-based optimization algorithm.
- Particles learn from their own best (**pBest**) and the swarm's best (**gBest**).
- The process repeats until convergence



IMPLEMENTATION METHODOLOGY



PSO-Based Random Forest Hyperparameter Optimization



EXPECTED RESULTS

Expected outcomes of PSO-based Random Forest hyperparameter optimization

1



Improved / Maintained Performance

PSO is expected to identify Random Forest hyperparameter configurations that **improve or maintain** classification performance compared with the default Random Forest.



Improve or Maintain

2



Optimal Hyperparameters

PSO is expected to automatically identify an effective combination of Random Forest hyperparameters within the defined search space.



n_estimators



max_depth



min_samples_split

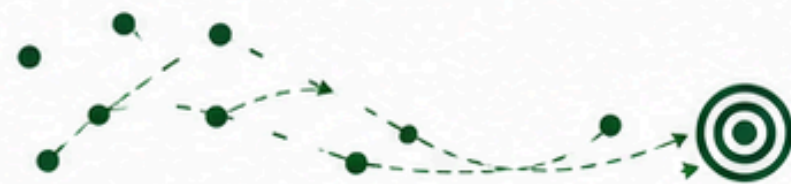
Automated search within predefined bounds

3



PSO Convergence

Particle fitness is expected to improve as the swarm explores promising regions of the hyperparameter search space.



INITIAL SWARM



FITNESS EVALUATION



pBest / gBest



PARTICLE UPDATE



BETTER SOLUTIONS



CONVERGENCE

4



Reliable & Reproducible Evaluation

The optimized configuration is expected to demonstrate consistent behavior across repeated experiments using controlled random seeds.



MULTIPLE PSO SEEDS



REPEATED EXPERIMENTS



PERFORMANCE STABILITY



Validation-guided optimization



EXPECTED OVERALL OUTCOME



RANDOM FOREST



PSO
HYPERPARAMETER
SEARCH



BEST
HYPERPARAMETER
CONFIGURATION



OPTIMIZED
RANDOM FOREST



PERFORMANCE
EVALUATION

PSO is expected to provide an **automated** and **systematic** approach for Random Forest hyperparameter optimization.



REFERENCES

01



[1] J. KENNEDY AND R. EBERHART, "PARTICLE SWARM OPTIMIZATION," IN PROC. IEEE INT. CONF. NEURAL NETWORKS (ICNN), PERTH, WA, AUSTRALIA, 1995, PP. 1942–1948.

02



[2] L. BREIMAN, "RANDOM FORESTS," MACHINE LEARNING, VOL. 45, NO. 1, PP. 5–32, 2001.

03



[3] D. DUA AND C. GRAFF, UCI MACHINE LEARNING REPOSITORY. IRVINE, CA, USA: UNIVERSITY OF CALIFORNIA, SCHOOL OF INFORMATION AND COMPUTER SCIENCES, 2019. [ONLINE]. AVAILABLE: [UCI MACHINE LEARNING REPOSITORY](https://archive.ics.uci.edu/)



THANK YOU